```
from google.colab import drive
drive.mount('/content/drive')
%cd drive/MyDrive/abcd/
%ls
```

```
Mounted at /content/drive
/content/drive/MyDrive/abcd
'DA Final AB_NYC_2019_updated NEW.csv'      room_type_vs_price.png
'DA Final AB_NYC_2019_updated NEW.gsheet'
total_price_by_neighbourhood_and_room_type.png
 price_by_neighbourhood_and_room_type.png
```

```
import pandas as pd

df = pd.read_csv('DA Final AB_NYC_2019_updated NEW.csv')

df.head()
```

```
     id  host_id    host_name neighbourhood_group neighbourhood
latitude  \
0  2539     2787         John            Brooklyn    Kensington
40.64749
1  2595     2845     Jennifer           Manhattan       Midtown
40.75362
2  3647     4632    Elisabeth           Manhattan        Harlem
40.80902
3  3831     4869  LisaRoxanne            Brooklyn  Clinton Hill
40.68514
4  5022     7192        Laura           Manhattan   East Harlem
40.79851

   longitude        room_type  price  minimum_nights
number_of_reviews  \
0  -73.97237     Private room    149               1
9
1  -73.98377  Entire home/apt    225               1
45
2  -73.94190     Private room    150               3
0
3  -73.95976  Entire home/apt     89               1
270
4  -73.94399  Entire home/apt     80              10
9

   reviews_per_month  calculated_host_listings_count  availability_365

0               0.21                               6               365

1               0.38                               2               355

2               0.00                               1               365
```

```
3                4.64                                1                194

4                0.10                                1                  0
```

```
df.isnull().sum()
```

```
id                                  0
host_id                             0
host_name                          21
neighbourhood_group                 0
neighbourhood                       0
latitude                            0
longitude                           0
room_type                           0
price                               0
minimum_nights                      0
number_of_reviews                   0
reviews_per_month                   0
calculated_host_listings_count      0
availability_365                    0
dtype: int64
```

```
df.head(20)
```

```
      id  host_id         host_name neighbourhood_group
neighbourhood  \
0   2539     2787              John            Brooklyn
Kensington
1   2595     2845          Jennifer           Manhattan
Midtown
2   3647     4632         Elisabeth           Manhattan
Harlem
3   3831     4869       LisaRoxanne            Brooklyn        Clinton
Hill
4   5022     7192             Laura           Manhattan         East
Harlem
5   5099     7322             Chris           Manhattan         Murray
Hill
6   5121     7356             Garon            Brooklyn  Bedford-
Stuyvesant
7   5178     8967          Shunichi           Manhattan      Hell's
Kitchen
8   5203     7490         MaryEllen           Manhattan     Upper West
Side
9   5238     7549               Ben           Manhattan
Chinatown
10  5295     7702              Lena           Manhattan     Upper West
Side
11  5441     7989              Kate           Manhattan      Hell's
```

```
Kitchen
12  5803     9744            Laurie            Brooklyn         South
Slope
13  6021    11528           Claudio           Manhattan     Upper West
Side
14  6090    11975             Alina           Manhattan        West
Village
15  6848    15991     Allen & Irina            Brooklyn
Williamsburg
16  7097    17571              Jane            Brooklyn         Fort
Greene
17  7322    18946              Doti           Manhattan
Chelsea
18  7726    20950  Adam And Charity            Brooklyn       Crown
Heights
19  7750    17985              Sing           Manhattan         East
Harlem

    latitude  longitude        room_type  price  minimum_nights  \
0   40.64749  -73.97237     Private room    149               1
1   40.75362  -73.98377  Entire home/apt    225               1
2   40.80902  -73.94190     Private room    150               3
3   40.68514  -73.95976  Entire home/apt     89               1
4   40.79851  -73.94399  Entire home/apt     80              10
5   40.74767  -73.97500  Entire home/apt    200               3
6   40.68688  -73.95596     Private room     60              45
7   40.76489  -73.98493     Private room     79               2
8   40.80178  -73.96723     Private room     79               2
9   40.71344  -73.99037  Entire home/apt    150               1
10  40.80316  -73.96545  Entire home/apt    135               5
11  40.76076  -73.98867     Private room     85               2
12  40.66829  -73.98779     Private room     89               4
13  40.79826  -73.96113     Private room     85               2
14  40.73530  -74.00525  Entire home/apt    120              90
15  40.70837  -73.95352  Entire home/apt    140               2
16  40.69169  -73.97185  Entire home/apt    215               2
17  40.74192  -73.99501     Private room    140               1
18  40.67592  -73.94694  Entire home/apt     99               3
19  40.79685  -73.94872  Entire home/apt    190               7

    number_of_reviews  reviews_per_month
calculated_host_listings_count  \
0                   9               0.21
6
1                  45               0.38
2
2                   0               0.00
1
3                 270               4.64
```

```
1
4                     9                0.10
1
5                    74                0.59
1
6                    49                0.40
1
7                   430                3.47
1
8                   118                0.99
1
9                   160                1.33
4
10                   53                0.43
1
11                  188                1.50
1
12                  167                1.34
3
13                  113                0.91
1
14                   27                0.22
1
15                  148                1.20
1
16                  198                1.72
1
17                  260                2.12
1
18                   53                4.44
1
19                    0                0.00
2

    availability_365
0                365
1                355
2                365
3                194
4                  0
5                129
6                  0
7                220
8                  0
9                188
10                 6
11                39
12               314
13               333
```

```
14                  0
15                 46
16                321
17                 12
18                 21
19                249
```

DESCRIPTIVE STATISTICS

```
import pandas as pd

# Assuming you already have the DataFrame 'df' with the provided data

# Calculate mean
mean_price = df['price'].mean()

# Calculate median
median_price = df['price'].median()

# Calculate mode
mode_price = df['price'].mode().values[0]  # mode() returns a Series,
get the first value

# Calculate range
range_price = df['price'].max() - df['price'].min()

# Calculate standard deviation
std_dev_price = df['price'].std()

# Print the results
print(f"Mean Price: {mean_price}")
print(f"Median Price: {median_price}")
print(f"Mode Price: {mode_price}")
print(f"Range of Prices: {range_price}")
print(f"Standard Deviation of Prices: {std_dev_price}")
```

```
Mean Price: 152.7206871868289
Median Price: 106.0
Mode Price: 100
Range of Prices: 10000
Standard Deviation of Prices: 240.15416974718758
```

PRICE ANALYSIS

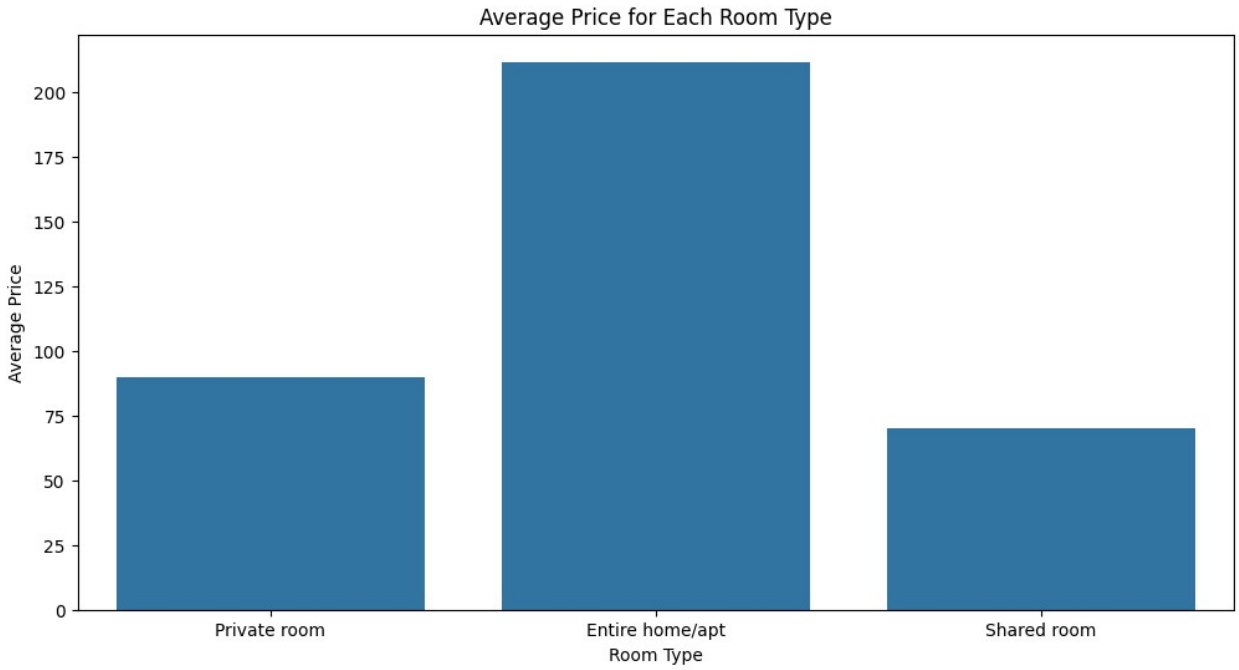
anova test by price and room type

```python
from scipy.stats import f_oneway

anova_result_room_types = f_oneway(
    df['price'][df['room_type'] == 'Private room'],
    df['price'][df['room_type'] == 'Entire home/apt'],
    df['price'][df['room_type'] == 'Shared room']
)

print("ANOVA for Room Types:")
print(anova_result_room_types)
```

```
ANOVA for Room Types:
F_onewayResult(statistic=1716.6257946347218, pvalue=0.0)
```

```python
import seaborn as sns
import matplotlib.pyplot as plt

# Assuming you have a DataFrame 'df' with your data

plt.figure(figsize=(12, 6))
sns.barplot(x='room_type', y='price', data=df, ci=None)  # ci=None to remove error bars
plt.title('Average Price for Each Room Type')
plt.xlabel('Room Type')
plt.ylabel('Average Price')
plt.show()
```

```
<ipython-input-16-b4ecc08c8a34>:7: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

  sns.barplot(x='room_type', y='price', data=df, ci=None)  # ci=None to remove error bars
```



t-test between brooklyn and manhattan

```
from scipy.stats import ttest_ind

# Example for neighbourhood groups
group1 = df['price'][df['neighbourhood_group'] == 'Brooklyn']
group2 = df['price'][df['neighbourhood_group'] == 'Manhattan']

result_ttest = ttest_ind(group1, group2)
print("T-Test for Neighbourhood Groups:")
print(result_ttest)
```

```
T-Test for Neighbourhood Groups:
TtestResult(statistic=-30.009360251152064, pvalue=8.973192961269122e-196, df=41763.0)
```

```
!pip install plotnine
```

```
Requirement already satisfied: plotnine in /usr/local/lib/python3.10/dist-packages (0.12.4)
Requirement already satisfied: matplotlib>=3.6.0 in /usr/local/lib/python3.10/dist-packages (from plotnine) (3.7.1)
Requirement already satisfied: mizani<0.10.0,>0.9.0 in /usr/local/lib/python3.10/dist-packages (from plotnine) (0.9.3)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.10/dist-packages (from plotnine) (1.23.5)
Requirement already satisfied: pandas>=1.5.0 in /usr/local/lib/python3.10/dist-packages (from plotnine) (1.5.3)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.10/dist-packages (from plotnine) (0.5.6)
```

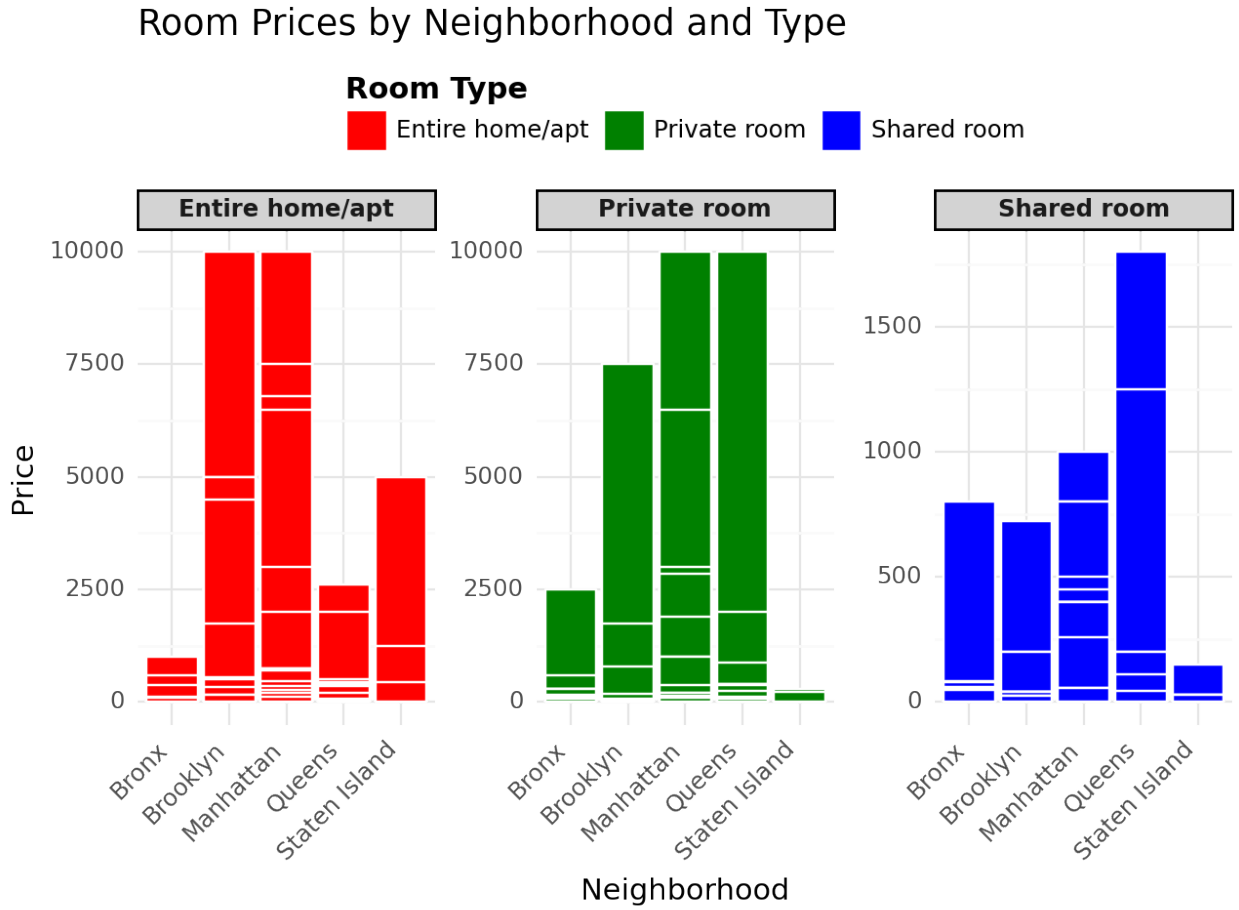
```
Requirement already satisfied: scipy>=1.5.0 in
/usr/local/lib/python3.10/dist-packages (from plotnine) (1.11.4)
Requirement already satisfied: statsmodels>=0.14.0 in
/usr/local/lib/python3.10/dist-packages (from plotnine) (0.14.1)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.10/dist-packages (from matplotlib>=3.6.0-
>plotnine) (1.2.0)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.10/dist-packages (from matplotlib>=3.6.0-
>plotnine) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.10/dist-packages (from matplotlib>=3.6.0-
>plotnine) (4.47.2)
Requirement already satisfied: kiwisolver>=1.0.1 in
/usr/local/lib/python3.10/dist-packages (from matplotlib>=3.6.0-
>plotnine) (1.4.5)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.10/dist-packages (from matplotlib>=3.6.0-
>plotnine) (23.2)
Requirement already satisfied: pillow>=6.2.0 in
/usr/local/lib/python3.10/dist-packages (from matplotlib>=3.6.0-
>plotnine) (9.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.10/dist-packages (from matplotlib>=3.6.0-
>plotnine) (3.1.1)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.10/dist-packages (from matplotlib>=3.6.0-
>plotnine) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.10/dist-packages (from pandas>=1.5.0->plotnine)
(2023.4)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-
packages (from patsy>=0.5.1->plotnine) (1.16.0)
```

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy
import seaborn as sns
import plotnine

from plotnine import ggplot, aes, geom_bar, facet_wrap, labs,
theme_minimal, theme, element_text, scale_fill_manual, element_rect

my_colors = ["red", "green", "blue"]
```

room price by neigbor hood and type

```
(ggplot(df, aes(x='neighbourhood_group', y='price', fill='room_type'))
+
 geom_bar(stat='identity', position='dodge', color='white', size=0.5)
```

```
+
 facet_wrap('~ room_type', scales='free_y', ncol=3) +
 labs(title='Room Prices by Neighborhood and Type',
      x='Neighborhood',
      y='Price',
      fill='Room Type') +
 theme_minimal() +
 theme(axis_text_x=element_text(angle=45, hjust=1),
       panel_grid_major=None,
       panel_grid_minor=None,
       legend_position='top',
       legend_title=element_text(face='bold'),
       legend_background=element_rect(fill='white', color='white'),
       legend_key=element_rect(color='white'),
       strip_background=element_rect(fill='lightgray'),
       strip_text=element_text(face='bold')) +
 scale_fill_manual(values=my_colors))
```




```
<Figure Size: (640 x 480)>
```